Rust Crash Course

> From zero to systems programmer — five levels, one document.

How to Use This Guide

Work through each level in order. Every section builds on the last. Run every code example yourself using:

```bash
cargo new playground && cd playground
# paste examples into src/main.rs, then:
cargo run
```

Install Rust if you haven't:

```bash
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Level 1 — Beginner

> Goals: Write basic programs, understand Rust's syntax, grasp ownership at a surface level.

1.1 Hello, World

```rust
fn main() {
    println!("Hello, world!");
}
```

`fn` declares a function. `main` is the entry point. `println!` is a **macro** (the `!` is the giveaway) — more on those later.

1.2 Variables and Mutability

In Rust, variables are **immutable by default**. This is a core safety feature.

```rust
```

```rust
fn main() {
    let x = 5;
    // x = 6; // ❌ This would not compile!

    let mut y = 5;
    y = 6; // ✅ Fine — y is mutable
    println!("x = {x}, y = {y}");
}
```

Shadowing — you can redeclare a variable with `let`, even changing its type:

```rust
fn main() {
    let spaces = "   ";         // &str
    let spaces = spaces.len(); // usize — different type, same name
    println!("{spaces}");
}
```

1.3 Data Types

Rust is **statically typed** — every value has a type known at compile time. The compiler infers types where it can.

Scalar Types

Type	Example	Notes
`i32`	`-42`	Signed 32-bit integer (default)
`u64`	`1000`	Unsigned 64-bit integer
`f64`	`3.14`	64-bit float (default)
`bool`	`true`	`true` or `false`
`char`	`'z'`	Unicode scalar, 4 bytes

```rust
```

```rust
fn main() {
    let n: i32 = -100;
    let big: u64 = 18_446_744_073_709_551_615; // underscores for readability
    let pi: f64 = 3.14159;
    let alive: bool = true;
    let letter: char = 'R';

    println!("{n} {big} {pi} {alive} {letter}");
}
```

Compound Types

Tuples — fixed-size, mixed types:

```rust
fn main() {
    let tup: (i32, f64, bool) = (42, 6.28, false);
    let (a, b, c) = tup;          // destructuring
    println!("{a} {} {}", tup.1, c); // index access with .0, .1, etc.
}
```

Arrays — fixed-size, same type:

```rust
fn main() {
    let arr = [1, 2, 3, 4, 5];
    let zeros = [0u8; 8]; // eight zeros of type u8
    println!("{}", arr[0]);
    println!("length: {}", arr.len());
}
```

> Arrays are stack-allocated and fixed in size. For growable lists, you use `Vec<T>` (covered soon).

1.4 Functions

```rust
```

```rust
fn main() {
    let result = add(3, 7);
    println!("3 + 7 = {result}");
}

fn add(a: i32, b: i32) -> i32 {
    a + b  // no semicolon = this is the return value (an "expression")
}
```

Key rules:

- Parameters **must** have explicit types
- Return type follows `->`
- The last expression in a function body is returned implicitly (no `return` needed, no semicolon)
- `return` exists but is only needed for early returns

```rust
fn classify(n: i32) -> &'static str {
    if n < 0 {
        return "negative"; // early return
    }
    if n == 0 { "zero" } else { "positive" }
}
```

1.5 Control Flow

if / else

```rust
```

```rust
fn main() {
    let temperature = 22;

    if temperature > 30 {
        println!("Hot");
    } else if temperature > 15 {
        println!("Comfortable");
    } else {
        println!("Cold");
    }

    // if is an expression — it returns a value
    let description = if temperature > 20 { "warm" } else { "cool" };
    println!("It's {description}");
}
```

loop

```rust
fn main() {
    let mut count = 0;
    let result = loop {
        count += 1;
        if count == 5 {
            break count * 2; // loop can return a value
        }
    };
    println!("result: {result}"); // 10
}
```

while

```rust
fn main() {
    let mut n = 3;
    while n > 0 {
        println!("{n}!");
        n -= 1;
    }
    println!("Liftoff!");
}
```

```rust
fn main() {
    // iterate over a range
    for i in 0..5 {
        print!("{i} ");
    }
    println!();

    // iterate over a collection
    let fruits = ["apple", "banana", "cherry"];
    for fruit in &fruits {
        println!("{fruit}");
    }
}
```

1.6 Ownership — The Big Idea

Ownership is Rust's central innovation. It replaces garbage collection with a set of compile-time rules that guarantee memory safety.

Three rules:

1. Each value has exactly one owner
2. When the owner goes out of scope, the value is dropped (freed)
3. There can only be one owner at a time

```rust
fn main() {
    let s1 = String::from("hello"); // s1 owns the string
    let s2 = s1;                    // ownership moves to s2
    // println!("{s1}"); ❌ s1 is no longer valid!
    println!("{s2}"); // ✅
}
```

Clone to make a deep copy if you need both:

```rust
```

```rust
fn main() {
    let s1 = String::from("hello");
    let s2 = s1.clone(); // explicit deep copy
    println!("{s1} and {s2}"); // ✅ both valid
}
```

Copy types (integers, bools, chars, tuples of Copy types) are copied automatically — no move:

```rust
fn main() {
    let x = 5;
    let y = x; // x is copied, not moved
    println!("{x} and {y}"); // ✅ both fine
}
```

1.7 References and Borrowing

Instead of transferring ownership, you can **borrow** a value with a reference `&`:

```rust
fn main() {
    let s = String::from("hello");
    let len = calculate_length(&s); // borrow s, don't move it
    println!("'{s}' has length {len}"); // s still valid
}

fn calculate_length(s: &String) -> usize {
    s.len()
} // s goes out of scope but doesn't drop anything — it's just a reference
```

Mutable references — borrow and modify:

```rust
```

```rust
fn main() {
    let mut s = String::from("hello");
    append_world(&mut s);
    println!("{s}"); // "hello, world"
}

fn append_world(s: &mut String) {
    s.push_str(", world");
}
```

The borrow rules (enforced at compile time):

- You can have **many immutable** references OR **one mutable** reference — never both simultaneously
- References must always be valid (no dangling pointers)

```rust
fn main() {
    let mut s = String::from("hello");
    let r1 = &s;
    let r2 = &s;
    println!("{r1} and {r2}"); // ✅ two immutable refs are fine

    let r3 = &mut s; // ✅ fine — r1 and r2 are no longer used after this point
    println!("{r3}");
}
```

1.8 Strings

Rust has two string types:

Type	Stored	Mutable	Owned
`&str`	In binary / stack	No	No (a reference)
`String`	Heap	Yes	Yes

```rust
```

```rust
fn main() {
    let literal: &str = "I am a string slice";
    let owned: String = String::from("I am heap-allocated");
    let also_owned: String = "I am too".to_string();

    // Convert &str -> String
    let s: String = literal.to_string();

    // Convert String -> &str
    let slice: &str = &owned;

    println!("{literal}");
    println!("{owned} — length {}", owned.len());

    // Common operations
    let mut s = String::from("Hello");
    s.push(' ');
    s.push_str("world");
    println!("{s}");

    // String formatting
    let name = "Ross";
    let greeting = format!("Hello, {name}!");
    println!("{greeting}");
}
```

1.9 Vectors

`Vec<T>` is a growable, heap-allocated array:

```rust
```

```rust
fn main() {
    let mut v: Vec<i32> = Vec::new();
    v.push(1);
    v.push(2);
    v.push(3);

    // shorthand macro
    let v2 = vec![10, 20, 30];

    // access
    let third = &v[2];
    println!("Third element: {third}");

    // safe access — returns Option
    match v.get(10) {
        Some(val) => println!("Got {val}"),
        None => println!("Index out of bounds"),
    }

    // iterate
    for n in &v {
        println!("{n}");
    }

    println!("Length: {}", v.len());
}
```

Level 1 — Challenge Projects

- **FizzBuzz**: Print 1–100, replacing multiples of 3 with "Fizz", 5 with "Buzz", both with "FizzBuzz"
- **Temperature converter**: Function that converts Celsius to Fahrenheit
- **Fibonacci**: Return the nth Fibonacci number using a loop
- **Simple calculator**: Accept two numbers and an operator via CLI args, return the result

---

Level 2 — Intermediate

> Goals: Model data with structs and enums, handle errors properly, use traits and generics.

2.1 Structs

Structs group related data together:

```rust
struct Server {
    hostname: String,
    ip: String,
    port: u16,
    online: bool,
}

impl Server {
    // associated function (like a static method / constructor)
    fn new(hostname: &str, ip: &str, port: u16) -> Self {
        Server {
            hostname: hostname.to_string(),
            ip: ip.to_string(),
            port,          // field init shorthand when var name == field name
            online: false,
        }
    }

    // method — takes &self (immutable borrow of the instance)
    fn address(&self) -> String {
        format!("{}:{}", self.ip, self.port)
    }

    // method that mutates
    fn connect(&mut self) {
        self.online = true;
        println!("{} is now online", self.hostname);
    }
}

fn main() {
    let mut srv = Server::new("web-01", "10.0.0.1", 8080);
    srv.connect();
    println!("Address: {}", srv.address());
}
```

Tuple structs — named tuples:

```rust
```

```rust
struct Colour(u8, u8, u8);
struct Metres(f64);

fn main() {
    let red = Colour(255, 0, 0);
    let distance = Metres(1.5);
    println!("Red: {}, {}, {}", red.0, red.1, red.2);
    println!("Distance: {}m", distance.0);
}
```

Struct update syntax:

```rust
struct Point { x: f64, y: f64, z: f64 }

fn main() {
    let origin = Point { x: 0.0, y: 0.0, z: 0.0 };
    let above = Point { z: 5.0, ..origin }; // copy remaining fields from origin
    println!("{} {} {}", above.x, above.y, above.z);
}
```

2.2 Enums

Enums are far more powerful in Rust than in most languages — each variant can carry data:

```rust
```

```rust
enum IpAddr {
    V4(u8, u8, u8, u8),
    V6(String),
}

fn main() {
    let home = IpAddr::V4(127, 0, 0, 1);
    let remote = IpAddr::V6(String::from("::1"));

    describe(&home);
    describe(&remote);
}

fn describe(ip: &IpAddr) {
    match ip {
        IpAddr::V4(a, b, c, d) => println!("IPv4: {a}.{b}.{c}.{d}"),
        IpAddr::V6(addr) => println!("IPv6: {addr}"),
    }
}
```

Enums with struct-like variants:

```rust
enum Event {
    Login { user: String, timestamp: u64 },
    Logout { user: String },
    Error(String),
}
```

2.3 Pattern Matching

`match` is exhaustive — the compiler ensures every case is handled:

```rust
```

```rust
fn describe_number(n: i32) -> &'static str {
    match n {
        0 => "zero",
        1 | 2 | 3 => "small",
        4..=9 => "medium",       // inclusive range
        n if n < 0 => "negative", // guard condition
        _ => "large",            // catch-all
    }
}

fn main() {
    for n in [-5, 0, 2, 6, 100] {
        println!("{n}: {}", describe_number(n));
    }
}
```

Destructuring in match:

```rust
struct Point { x: i32, y: i32 }

fn main() {
    let p = Point { x: 3, y: -5 };
    match p {
        Point { x: 0, y } => println!("On y-axis at {y}"),
        Point { x, y: 0 } => println!("On x-axis at {x}"),
        Point { x, y }    => println!("At ({x}, {y})"),
    }
}
```

`if let` — concise match when you only care about one case:

```rust
fn main() {
    let number = Some(7);
    if let Some(n) = number {
        println!("Got {n}");
    }
}
```

2.4 Option<T>

Rust has no `null`. Instead, optional values use `Option<T>`:

```rust
enum Option<T> {
    Some(T),
    None,
}
```

```rust
fn find_user(id: u32) -> Option<String> {
    if id == 1 {
        Some("alice".to_string())
    } else {
        None
    }
}

fn main() {
    // match
    match find_user(1) {
        Some(name) => println!("Found: {name}"),
        None => println!("Not found"),
    }

    // unwrap_or — provide a default
    let name = find_user(99).unwrap_or("anonymous".to_string());
    println!("{name}");

    // ? operator (more on this in error handling)
    // map — transform the inner value if Some
    let upper = find_user(1).map(|s| s.to_uppercase());
    println!("{:?}", upper);
}
```

2.5 Error Handling with Result<T, E>

`Result` is used for operations that can fail:

```rust
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

```rust
use std::num::ParseIntError;

fn parse_port(s: &str) -> Result<u16, ParseIntError> {
    let n: u16 = s.trim().parse()?; // ? propagates the error if Err
    Ok(n)
}

fn main() {
    match parse_port("8080") {
        Ok(port) => println!("Port: {port}"),
        Err(e) => println!("Error: {e}"),
    }

    match parse_port("not-a-number") {
        Ok(port) => println!("Port: {port}"),
        Err(e) => println!("Error: {e}"),
    }
}
```

The `?` operator — inside a function returning `Result`, `?` either unwraps `Ok` or returns the `Err` to the caller immediately. It eliminates most match boilerplate.

```rust
use std::fs;

fn read_config(path: &str) -> Result<String, std::io::Error> {
    let contents = fs::read_to_string(path)?; // propagate if error
    Ok(contents.to_uppercase())
}

fn main() {
    match read_config("/etc/hostname") {
        Ok(s) => println!("{s}"),
        Err(e) => eprintln!("Failed: {e}"),
    }
}
```

2.6 Traits

Traits define shared behaviour — Rust's equivalent of interfaces:

```rust
```

```rust
trait Describe {
    fn describe(&self) -> String;

    // default implementation — can be overridden
    fn shout(&self) -> String {
        self.describe().to_uppercase()
    }
}

struct Dog { name: String }
struct Cat { name: String }

impl Describe for Dog {
    fn describe(&self) -> String {
        format!("{} is a dog", self.name)
    }
}

impl Describe for Cat {
    fn describe(&self) -> String {
        format!("{} is a cat", self.name)
    }
    // uses default shout()
}

fn print_info(animal: &impl Describe) {
    println!("{}", animal.describe());
    println!("{}", animal.shout());
}

fn main() {
    let d = Dog { name: "Rex".to_string() };
    let c = Cat { name: "Whiskers".to_string() };
    print_info(&d);
    print_info(&c);
}
```

Common standard library traits you'll use constantly:

- `Display` — `{}` formatting
- `Debug` — `{:?}` formatting (derive with `#[derive(Debug)]`)
- `Clone` — `.clone()`
- `PartialEq` — `==` comparison
- `Iterator` — gives you the entire iterator adapter chain

- `From` / `Into` — type conversions

```rust
#[derive(Debug, Clone, PartialEq)]
struct Config {
    host: String,
    port: u16,
}

fn main() {
    let c1 = Config { host: "localhost".to_string(), port: 8080 };
    let c2 = c1.clone();
    println!("{:?}", c1);
    println!("Equal: {}", c1 == c2);
}
```

2.7 Generics

Write code that works for many types without duplication:

```rust
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut largest = &list[0];
    for item in list {
        if item > largest {
            largest = item;
        }
    }
    largest
}

fn main() {
    let numbers = vec![34, 50, 25, 100, 65];
    println!("Largest: {}", largest(&numbers));

    let chars = vec!['y', 'm', 'a', 'q'];
    println!("Largest: {}", largest(&chars));
}
```

Generic structs:

```rust
```

```rust
struct Pair<T> {
    first: T,
    second: T,
}

impl<T: std::fmt::Display> Pair<T> {
    fn show(&self) {
        println!("({}, {})", self.first, self.second);
    }
}

fn main() {
    let p = Pair { first: 5, second: 10 };
    p.show();
}
```

2.8 HashMaps

```rust
```

```rust
use std::collections::HashMap;

fn main() {
    let mut scores: HashMap<String, u32> = HashMap::new();

    scores.insert("Alice".to_string(), 95);
    scores.insert("Bob".to_string(), 78);

    // get returns Option<&V>
    if let Some(score) = scores.get("Alice") {
        println!("Alice: {score}");
    }

    // insert only if not present
    scores.entry("Charlie".to_string()).or_insert(88);

    // iterate
    for (name, score) in &scores {
        println!("{name}: {score}");
    }

    // update based on existing value
    let count = scores.entry("Bob".to_string()).or_insert(0);
    *count += 10;
    println!("Bob updated: {}", scores["Bob"]);
}
```

Level 2 — Challenge Projects

- **Contact book**: Store contacts (name, email, phone) in a `Vec<Struct>`, support add/search/remove
- **Config file parser**: Parse a simple `key=value` format into a `HashMap`
- **Error-safe file reader**: Read lines from a file, skip malformed ones, return results
- **Shape area calculator**: Enum with Circle, Rectangle, Triangle variants, each with an `area()` method via a trait

---

Level 3 — Adept

> Goals: Understand lifetimes, master iterators and closures, organise code with modules, and use the ecosystem effectively.

3.1 Lifetimes

Lifetimes tell the compiler how long references are valid. Usually inferred, but sometimes you need to be explicit:

```rust
// Without lifetime annotation this would not compile
// The compiler needs to know: does the returned reference live as long as x or y?
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("long string");
    let result;
    {
        let s2 = String::from("xyz");
        result = longest(s1.as_str(), s2.as_str());
        println!("Longest: {result}"); // ✅ result used while both are alive
    }
}
```

Lifetimes in structs — when a struct holds a reference:

```rust
struct Excerpt<'a> {
    text: &'a str,
}

impl<'a> Excerpt<'a> {
    fn content(&self) -> &str {
        self.text
    }
}

fn main() {
    let text = String::from("Call me Ishmael. Some years ago...");
    let first = text.split('.').next().unwrap();
    let e = Excerpt { text: first };
    println!("{}", e.content());
}
```

> **Tip**: If the borrow checker complains about lifetimes, first try to restructure your code to own data rather than reference it (`String` instead of `&str`). Explicit lifetimes are often

> avoidable.

3.2 Closures

Closures are anonymous functions that can capture variables from their enclosing scope:

```rust
fn main() {
    let multiplier = 3;

    // closure captures `multiplier` from the environment
    let multiply = |x| x * multiplier;

    println!("{}", multiply(5));  // 15
    println!("{}", multiply(10)); // 30

    // closures as function parameters
    let numbers = vec![1, 2, 3, 4, 5];
    let evens: Vec<i32> = numbers.iter().filter(|&&n| n % 2 == 0).cloned().collect()
    println!("{:?}", evens);
}
```

Closure traits:

- `Fn` — borrows immutably, can be called many times
- `FnMut` — borrows mutably, can be called many times
- `FnOnce` — takes ownership, can only be called once

```rust
fn apply<F: Fn(i32) -> i32>(f: F, value: i32) -> i32 {
    f(value)
}

fn main() {
    let double = |x| x * 2;
    let square = |x: i32| x.pow(2);
    println!("{}", apply(double, 5));  // 10
    println!("{}", apply(square, 5)); // 25
}
```

`move` **closures** — force ownership into the closure (needed for threads):

```rust
fn main() {
    let text = String::from("hello");
    let print_it = move || println!("{text}"); // text is moved into the closure
    print_it();
    // println!("{text}"); ❌ text was moved
}
```

3.3 Iterators

Iterators are lazy — they do no work until consumed. Chain adapters to build expressive pipelines:

```rust
```

```rust
fn main() {
    let v = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

    // filter → map → collect
    let result: Vec<i32> = v.iter()
        .filter(|&&n| n % 2 == 0)   // keep evens
        .map(|&n| n * n)             // square them
        .collect();
    println!("{:?}", result); // [4, 16, 36, 64, 100]

    // sum, product
    let total: i32 = v.iter().sum();
    println!("Sum: {total}");

    // any / all
    let has_big = v.iter().any(|&n| n > 8);
    println!("Has number > 8: {has_big}");

    // enumerate
    for (i, val) in v.iter().enumerate() {
        if i < 3 { println!("[{i}] = {val}"); }
    }

    // flat_map
    let words = vec!["hello world", "foo bar"];
    let all_words: Vec<&str> = words.iter()
        .flat_map(|s| s.split_whitespace())
        .collect();
    println!("{:?}", all_words);

    // chaining multiple sources
    let a = vec![1, 2, 3];
    let b = vec![4, 5, 6];
    let combined: Vec<i32> = a.iter().chain(b.iter()).cloned().collect();
    println!("{:?}", combined);
}
```

Implementing Iterator yourself:

```rust
```

```rust
struct Counter {
    count: u32,
    max: u32,
}

impl Counter {
    fn new(max: u32) -> Self {
        Counter { count: 0, max }
    }
}

impl Iterator for Counter {
    type Item = u32;

    fn next(&mut self) -> Option<Self::Item> {
        if self.count < self.max {
            self.count += 1;
            Some(self.count)
        } else {
            None
        }
    }
}

fn main() {
    let sum: u32 = Counter::new(5).sum();
    println!("Sum 1..=5: {sum}"); // 15
}
```

3.4 Modules and Project Structure

Rust organises code with modules:

```rust
```

```rust
// src/main.rs
mod network {
    pub mod server {
        pub struct Server {
            pub port: u16,
        }

        impl Server {
            pub fn new(port: u16) -> Self {
                Server { port }
            }

            pub fn start(&self) {
                println!("Server listening on port {}", self.port);
            }
        }
    }

    pub fn status() -> &'static str {
        "online"
    }
}

use network::server::Server;

fn main() {
    let srv = Server::new(8080);
    srv.start();
    println!("Status: {}", network::status());
}
```

Multi-file modules — for real projects:

```
src/
  main.rs
  config.rs        ← mod config; in main.rs
  network/
    mod.rs         ← mod network; in main.rs
    server.rs      ← mod server; in network/mod.rs
```

3.5 Cargo and the Ecosystem

Key `Cargo.toml` sections:

```toml
[package]
name = "my-tool"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { version = "1", features = ["derive"] }
serde_json = "1"
tokio = { version = "1", features = ["full"] }
anyhow = "1"
reqwest = { version = "0.12", features = ["json"] }

[dev-dependencies]
pretty_assertions = "1"
```

Essential crates for your level:

Crate	Purpose
`serde` + `serde_json`	Serialisation/deserialisation
`anyhow`	Easy error handling
`tokio`	Async runtime
`reqwest`	HTTP client
`clap`	CLI argument parsing
`log` + `env_logger`	Logging
`chrono`	Date/time
`regex`	Regular expressions

3.6 Error Handling with anyhow

For application code (not libraries), `anyhow` makes error handling much nicer:

```rust
```

```rust
use anyhow::{Context, Result, bail};

fn read_port(s: &str) -> Result<u16> {
    let n: u16 = s.parse().context("Port must be a number")?;
    if n < 1024 {
        bail!("Port {n} is reserved (must be >= 1024)");
    }
    Ok(n)
}

fn main() -> Result<()> {
    let port = read_port("8080")?;
    println!("Using port {port}");
    Ok(())
}
```

3.7 Closures + Iterators in Practice

Here's a more realistic example — parsing and processing log lines:

```rust
```

```rust
#[derive(Debug)]
struct LogEntry {
    level: String,
    message: String,
}

fn parse_line(line: &str) -> Option<LogEntry> {
    let parts: Vec<&str> = line.splitn(2, ' ').collect();
    if parts.len() != 2 { return None; }
    Some(LogEntry {
        level: parts[0].to_string(),
        message: parts[1].to_string(),
    })
}

fn main() {
    let raw_logs = vec![
        "ERROR disk full",
        "INFO service started",
        "WARN memory high",
        "bad line",
        "ERROR connection timeout",
    ];

    let errors: Vec<LogEntry> = raw_logs.iter()
        .filter_map(|line| parse_line(line))
        .filter(|e| e.level == "ERROR")
        .collect();

    for e in &errors {
        println!("[{}] {}", e.level, e.message);
    }
    println!("Total errors: {}", errors.len());
}
```

Level 3 — Challenge Projects

- **CLI tool with clap:** Build a port scanner or ping tool with proper argument parsing
- **JSON config loader:** Read a JSON config file using `serde`, apply defaults for missing fields
- **Log analyser:** Read a log file, count entries by level, find the most common error messages
- **Custom iterator:** Build a `PagedIterator` that yields items in chunks

---

Level 4 — Expert

> Goals: Async programming with Tokio, smart pointers, dynamic dispatch, advanced generics, threading.

4.1 Async / Await and Tokio

Async Rust lets you write concurrent code without threads for every task. The most common runtime is Tokio.

```toml
# Cargo.toml
[dependencies]
tokio = { version = "1", features = ["full"] }
```

```rust
use tokio::time::{sleep, Duration};

#[tokio::main]
async fn main() {
    println!("Starting tasks...");

    // run two async tasks concurrently
    let (r1, r2) = tokio::join!(
        fetch_data("service-a"),
        fetch_data("service-b"),
    );

    println!("{r1}");
    println!("{r2}");
}

async fn fetch_data(source: &str) -> String {
    sleep(Duration::from_millis(100)).await;
    format!("Data from {source}")
}
```

Spawning tasks (truly concurrent, on the thread pool):

```rust
```

```rust
use tokio::task;

#[tokio::main]
async fn main() {
    let handle = task::spawn(async {
        // runs concurrently
        expensive_work().await
    });

    do_other_things().await;

    let result = handle.await.unwrap();
    println!("{result}");
}

async fn expensive_work() -> String {
    tokio::time::sleep(tokio::time::Duration::from_secs(1)).await;
    "done".to_string()
}

async fn do_other_things() {
    println!("Doing other things while task runs...");
}
```

Async channels:

```rust
use tokio::sync::mpsc;

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::<String>(32);

    tokio::spawn(async move {
        for i in 0..5 {
            tx.send(format!("message {i}")).await.unwrap();
        }
    });

    while let Some(msg) = rx.recv().await {
        println!("Received: {msg}");
    }
}
```

4.2 Smart Pointers

Box<T>

Heap-allocates a value. Useful for recursive types and large values:

```rust
// Recursive type — wouldn't compile without Box
enum List {
    Cons(i32, Box<List>),
    Nil,
}

fn main() {
    let list = List::Cons(1, Box::new(List::Cons(2, Box::new(List::Nil))));
}
```

Rc<T> — Reference Counted

Multiple ownership — for single-threaded code:

```rust
use std::rc::Rc;

fn main() {
    let data = Rc::new(vec![1, 2, 3]);
    let a = Rc::clone(&data); // not a deep clone — increments ref count
    let b = Rc::clone(&data);

    println!("Owners: {}", Rc::strong_count(&data)); // 3
    println!("{:?}", a);
    println!("{:?}", b);
}
```

Arc<T> — Atomic Reference Counted

Like `Rc` but thread-safe:

```rust
```

```rust
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3, 4, 5]);
    let mut handles = vec![];

    for i in 0..3 {
        let data = Arc::clone(&data);
        handles.push(thread::spawn(move || {
            println!("Thread {i} sees: {:?}", data);
        }));
    }

    for h in handles { h.join().unwrap(); }
}
```

Mutex<T> — Mutual Exclusion

Safely mutate shared data across threads:

```rust
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0u32));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            let mut n = counter.lock().unwrap();
            *n += 1;
        }));
    }

    for h in handles { h.join().unwrap(); }
    println!("Final count: {}", *counter.lock().unwrap()); // 10
}
```

4.3 Trait Objects and Dynamic Dispatch

When you need a collection of different types that share a trait:

```rust
trait Plugin {
    fn name(&self) -> &str;
    fn execute(&self, input: &str) -> String;
}

struct UppercasePlugin;
struct ReversePlugin;

impl Plugin for UppercasePlugin {
    fn name(&self) -> &str { "uppercase" }
    fn execute(&self, input: &str) -> String { input.to_uppercase() }
}

impl Plugin for ReversePlugin {
    fn name(&self) -> &str { "reverse" }
    fn execute(&self, input: &str) -> String { input.chars().rev().collect() }
}

fn run_pipeline(plugins: &[Box<dyn Plugin>], input: &str) -> String {
    plugins.iter().fold(input.to_string(), |acc, p| p.execute(&acc))
}

fn main() {
    let plugins: Vec<Box<dyn Plugin>> = vec![
        Box::new(UppercasePlugin),
        Box::new(ReversePlugin),
    ];

    let result = run_pipeline(&plugins, "hello");
    println!("{result}"); // OLLEH
}
```

`impl Trait` vs `dyn Trait`:

- `impl Trait` — static dispatch, resolved at compile time, faster, but each caller gets its own monomorphised copy
- `dyn Trait` — dynamic dispatch, runtime vtable lookup, slight overhead, but allows heterogeneous collections

---

4.4 Advanced Generics and Trait Bounds

```rust
use std::fmt::{Debug, Display};

// multiple bounds
fn print_info<T: Debug + Display + Clone>(value: T) {
    println!("Display: {value}");
    println!("Debug: {value:?}");
    let _copy = value.clone();
}

// where clause — cleaner for complex bounds
fn complex<T, U>(t: T, u: U) -> String
where
    T: Display + Clone,
    U: Debug,
{
    format!("{t} / {u:?}")
}

// returning impl Trait
fn make_adder(x: i32) -> impl Fn(i32) -> i32 {
    move |y| x + y
}

fn main() {
    print_info(42);
    print_info("hello");
    let add5 = make_adder(5);
    println!("{}", add5(3)); // 8
}
```

4.5 Iterators — Going Deeper

Custom iterator adapters:

```rust
```

```rust
struct Throttle<I: Iterator> {
    inner: I,
    every: usize,
    count: usize,
}

impl<I: Iterator> Iterator for Throttle<I> {
    type Item = I::Item;

    fn next(&mut self) -> Option<Self::Item> {
        loop {
            self.count += 1;
            let item = self.inner.next()?;
            if self.count % self.every == 0 {
                return Some(item);
            }
        }
    }
}

trait ThrottleExt: Iterator + Sized {
    fn throttle(self, every: usize) -> Throttle<Self> {
        Throttle { inner: self, every, count: 0 }
    }
}

impl<I: Iterator> ThrottleExt for I {}

fn main() {
    let v: Vec<i32> = (1..=20).throttle(5).collect();
    println!("{:?}", v); // [5, 10, 15, 20]
}
```

4.6 Error Handling in Libraries — thiserror

For library code, define your own error types:

```rust
```

```rust
use thiserror::Error;

#[derive(Debug, Error)]
enum AppError {
    #[error("IO error: {0}")]
    Io(#[from] std::io::Error),

    #[error("Parse error: expected {expected}, got {got}")]
    Parse { expected: String, got: String },

    #[error("Not found: {0}")]
    NotFound(String),
}

fn find_config(name: &str) -> Result<String, AppError> {
    if name.is_empty() {
        return Err(AppError::NotFound("name cannot be empty".to_string()));
    }
    Ok(format!("config for {name}"))
}

fn main() {
    match find_config("") {
        Ok(c) => println!("{c}"),
        Err(e) => eprintln!("Error: {e}"),
    }
}
```

Level 4 — Challenge Projects

- **Async HTTP server:** Use `axum` to build a REST API with multiple routes and JSON responses
- **Concurrent file downloader:** Download N files in parallel using `tokio::spawn` and `reqwest`
- **Plugin system:** Dynamic dispatch with `Box<dyn Trait>` — load different "processors" at runtime
- **Thread-safe cache:** `Arc<Mutex<HashMap<K,V>>>` with expiry logic

Level 5 — Master

> Goals: Unsafe Rust, macros, FFI, zero-cost abstractions, performance tuning, and advanced

> patterns.

5.1 Unsafe Rust

`unsafe` unlocks operations the borrow checker cannot verify. Use it sparingly and always document why it's sound:

```rust
fn main() {
    // Raw pointers
    let mut x = 42i32;
    let raw = &mut x as *mut i32;

    unsafe {
        *raw = 100; // dereference raw pointer
        println!("{}", *raw);
    }

    // Calling an unsafe function
    unsafe {
        dangerous();
    }
}

unsafe fn dangerous() {
    println!("You called an unsafe function");
}
```

Unsafe superpowers:

1. Dereference raw pointers
2. Call unsafe functions or C FFI
3. Access/modify mutable static variables
4. Implement unsafe traits
5. Access fields of `union`s

Wrapping unsafe in a safe API — this is the real goal:

```rust
```

```rust
pub fn split_at(slice: &[i32], mid: usize) -> (&[i32], &[i32]) {
    let len = slice.len();
    let ptr = slice.as_ptr();
    assert!(mid <= len);

    // SAFETY: mid <= len guarantees both slices are within bounds
    unsafe {
        (
            std::slice::from_raw_parts(ptr, mid),
            std::slice::from_raw_parts(ptr.add(mid), len - mid),
        )
    }
}

fn main() {
    let v = vec![1, 2, 3, 4, 5];
    let (left, right) = split_at(&v, 2);
    println!("{:?} {:?}", left, right);
}
```

5.2 Foreign Function Interface (FFI)

Call C code from Rust:

```rust
// Link to libc's abs function
extern "C" {
    fn abs(input: i32) -> i32;
}

fn main() {
    unsafe {
        println!("abs(-5) = {}", abs(-5));
    }
}
```

Expose Rust to C:

```rust
```

```
#[no_mangle]
pub extern "C" fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

For serious FFI work, use the `bindgen` crate to auto-generate bindings from C headers.

5.3 Declarative Macros

`macro_rules!` lets you write code that writes code:

```rust
macro_rules! log_info {
    ($($arg:tt)*) => {
        println!("[INFO] {}", format!($($arg)*));
    };
}

macro_rules! hashmap {
    ($($k:expr => $v:expr),* $(,)?) => {{
        let mut map = std::collections::HashMap::new();
        $(map.insert($k, $v);)*
        map
    }};
}

fn main() {
    log_info!("Server started on port {}", 8080);

    let m = hashmap! {
        "alice" => 95,
        "bob" => 78,
    };
    println!("{:?}", m);
}
```

5.4 Procedural Macros

Proc macros operate on the AST. The most common is derive macros — like `serde`'s `#[derive(Serialize, Deserialize)]`.

Writing your own requires a separate crate. Here's a minimal example of what one looks like:

```rust
// In a proc-macro crate:
use proc_macro::TokenStream;
use quote::quote;
use syn;

#[proc_macro_derive(HelloMacro)]
pub fn hello_macro_derive(input: TokenStream) -> TokenStream {
    let ast = syn::parse(input).unwrap();
    let name = &ast.ident;
    let gen = quote! {
        impl HelloMacro for #name {
            fn hello() {
                println!("Hello from {}!", stringify!(#name));
            }
        }
    };
    gen.into()
}

// Usage in another crate:
// #[derive(HelloMacro)]
// struct MyStruct;
// MyStruct::hello(); // "Hello from MyStruct!"
```

5.5 Performance and Zero-Cost Abstractions

Zero-cost abstractions — Rust's iterators, generics, and async compile down to the same machine code as hand-written C. No runtime overhead.

Profile before optimising:

```bash
cargo build --release  # always benchmark release builds
cargo install flamegraph
cargo flamegraph --bin my-binary
```

Common performance techniques:

```rust
```

```rust
// Avoid allocations — use &str instead of String where possible
fn process(s: &str) -> usize { s.len() }

// Reuse allocations
fn batch_process(items: &[String]) -> Vec<usize> {
    let mut results = Vec::with_capacity(items.len()); // pre-allocate
    for item in items {
        results.push(item.len());
    }
    results
}

// Avoid cloning in iterators
fn sum_lengths(items: &[String]) -> usize {
    items.iter().map(|s| s.len()).sum()
    // .iter() borrows — no clone needed
}

// Use iterators over manual loops — compiler can auto-vectorise
fn sum_squares(v: &[f64]) -> f64 {
    v.iter().map(|x| x * x).sum()
}
```

`Cow<str>` — clone on write, borrow when possible:

```rust
use std::borrow::Cow;

fn normalise(s: &str) -> Cow<str> {
    if s.contains(' ') {
        Cow::Owned(s.replace(' ', "_"))
    } else {
        Cow::Borrowed(s) // zero allocation
    }
}

fn main() {
    println!("{}", normalise("hello world")); // owned
    println!("{}", normalise("nospaces"));    // borrowed
}
```

5.6 Advanced Trait Patterns

Newtype Pattern

```rust
struct Metres(f64);
struct Kilograms(f64);

// You can't accidentally pass Kilograms where Metres expected
fn move_object(distance: Metres, weight: Kilograms) {
    println!("Moving {}kg by {}m", weight.0, distance.0);
}
```

Typestate Pattern — encode state in the type system

```rust
```

```rust
struct Connection<State> {
    host: String,
    _state: std::marker::PhantomData<State>,
}

struct Disconnected;
struct Connected;

impl Connection<Disconnected> {
    fn new(host: &str) -> Self {
        Connection { host: host.to_string(), _state: Default::default() }
    }

    fn connect(self) -> Connection<Connected> {
        println!("Connecting to {}", self.host);
        Connection { host: self.host, _state: Default::default() }
    }
}

impl Connection<Connected> {
    fn send(&self, data: &str) {
        println!("Sending '{}' to {}", data, self.host);
    }

    fn disconnect(self) -> Connection<Disconnected> {
        println!("Disconnecting");
        Connection { host: self.host, _state: Default::default() }
    }
}

fn main() {
    let conn = Connection::<Disconnected>::new("10.0.0.1")
        .connect();    // must connect before send()

    conn.send("hello");
    conn.disconnect();

    // conn.send("oops"); ❌ would not compile — conn was moved
}
```

Builder Pattern

```rust
```

```
#[derive(Debug)]
struct Request {
    url: String,
    method: String,
    timeout_ms: u64,
    headers: Vec<(String, String)>,
}

struct RequestBuilder {
    url: String,
    method: String,
    timeout_ms: u64,
    headers: Vec<(String, String)>,
}

impl RequestBuilder {
    fn new(url: &str) -> Self {
        RequestBuilder {
            url: url.to_string(),
            method: "GET".to_string(),
            timeout_ms: 5000,
            headers: vec![],
        }
    }

    fn method(mut self, m: &str) -> Self { self.method = m.to_string(); self }
    fn timeout(mut self, ms: u64) -> Self { self.timeout_ms = ms; self }
    fn header(mut self, k: &str, v: &str) -> Self {
        self.headers.push((k.to_string(), v.to_string())); self
    }

    fn build(self) -> Request {
        Request {
            url: self.url,
            method: self.method,
            timeout_ms: self.timeout_ms,
            headers: self.headers,
        }
    }
}

fn main() {
    let req = RequestBuilder::new("https://api.example.com/data")
        .method("POST")
        .timeout(10_000)
        .header("Authorization", "Bearer token123")
```

```
        .header("Content-Type", "application/json")
        .build();

    println!("{:?}", req);
}
```

5.7 Understanding the Borrow Checker — Interior Mutability

Sometimes you need mutability in a context where the borrow checker won't allow it. `RefCell<T>` moves borrow checking to runtime:

```rust
use std::cell::RefCell;

struct Logger {
    entries: RefCell<Vec<String>>,
}

impl Logger {
    fn new() -> Self {
        Logger { entries: RefCell::new(vec![]) }
    }

    // Takes &self but mutates internally
    fn log(&self, msg: &str) {
        self.entries.borrow_mut().push(msg.to_string());
    }

    fn print_all(&self) {
        for entry in self.entries.borrow().iter() {
            println!("{entry}");
        }
    }
}

fn main() {
    let logger = Logger::new();
    logger.log("Server started");
    logger.log("Connection received");
    logger.print_all();
}
```

> `RefCell` panics at runtime if you violate borrow rules. Use it only when you've proven safety that the compiler can't see.

5.8 Writing Idiomatic Rust

By the time you're here, these habits should be second nature:

```rust
// ✅ Use ? instead of match for error propagation
fn load(path: &str) -> anyhow::Result<String> {
    Ok(std::fs::read_to_string(path)?)
}

// ✅ Prefer iterators over index loops
fn sum_even(v: &[i32]) -> i32 {
    v.iter().filter(|&&n| n % 2 == 0).sum()
}

// ✅ Return Option/Result instead of sentinel values
fn first_even(v: &[i32]) -> Option<i32> {
    v.iter().find(|&&n| n % 2 == 0).copied()
}

// ✅ Use type aliases to simplify signatures
type Result<T> = std::result::Result<T, Box<dyn std::error::Error>>;

// ✅ Derive what you can
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
struct NodeId(u64);

// ✅ Destructure in function parameters
fn print_point(&(x, y): &(i32, i32)) {
    println!("({x}, {y})");
}
```

Level 5 — Challenge Projects

- **Build a macro:** Write a `vec_of_strings!["a", "b", "c"]` macro that produces a `Vec<String>`
- **Safe wrapper around raw syscall:** Use `unsafe` to call `getpid()` via FFI and wrap it in a safe API
- **Typestate state machine:** Model a TCP connection lifecycle with compile-time state enforcement
- **Async rate limiter:** Build a token bucket rate limiter using Tokio and `Arc<Mutex<>>`

- **Simple allocator**: Implement a basic memory allocator using a `static mut` byte array and unsafe

---

Reference Summary

Ownership Cheat Sheet

Situation	Solution
Need to use value in two places	`.clone()` or use references
Function needs to read a value	Pass `&T`
Function needs to modify a value	Pass `&mut T`
Multiple owners	`Rc<T>` (single-thread) or `Arc<T>` (multi-thread)
Shared mutation	`Arc<Mutex<T>>`
Interior mutability	`RefCell<T>`

Error Handling Cheat Sheet

Context	Tool
Application code	`anyhow::Result` + `?`
Library code	`thiserror` custom error enum
Quick scripts	`unwrap()` (acceptable for prototyping)
Recoverable optional	`Option<T>` + `?`

When to Use What

Need	Use
Growable list	`Vec<T>`
Key-value store	`HashMap<K, V>`
Unique values	`HashSet<T>`

Need	Use
Fixed-size list	`[T; N]` array
Heap-allocate one thing	`Box<T>`
Shared ownership	`Rc<T>` / `Arc<T>`
Dynamic dispatch	`Box<dyn Trait>`
String data you own	`String`
String data you borrow	`&str`
Possibly-absent value	`Option<T>`
Operation that can fail	`Result<T, E>`

Recommended Next Steps

1. **The Rust Book** — the official guide, free online, excellent
2. **Rustlings** — small exercises that fix broken code
3. **Rust by Example** — concept-per-page examples
4. **Jon Gjengset on YouTube** — expert-level Rust deep dives
5. **Zero to Production in Rust** — building a real web service end-to-end

---

Happy hacking. The borrow checker is not your enemy — it's your proof assistant.